



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 1: Các kiểu hệ thống thông tin

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Chương này tập trung trình bày một số chủ đề sau đây:

- 1.1. Khái niệm hệ thống thông tin
- 1.2. Các kiểu hệ thống hệ thống thông tin
- 1.3. Các cách tiếp cận phát triển hệ thống thông tin
- 1.4. Các pha cơ bản của phân tích và thiết kế
- 1.5. Các công nghệ cho phát triển các kiểu hệ thống phần mềm
- 1.6. Các quy trình phát triển phần mềm thông dụng

| 1.1 KHÁI NIỆM HỆ THỐNG THÔNG TIN

❖ Hệ thống là gì?

- Hệ thống là một tập hợp gồm nhiều phần tử, có mối quan hệ ràng buộc lẫn nhau và cùng hoạt động hướng tới một mục đích chung.
- Hệ thống nhận đầu vào (input) và xuất đầu ra (output).
- Ví dụ: HT triết học, HT luật pháp, HT tuần hoàn, HT bảo mật, HT thông tin ...

❖ Hệ thống thông tin (information system)?

- HTTT thường được sử dụng để chỉ một phạm trù liên quan đến máy tính bao gồm các phần tử như hệ phần mềm, các hệ phần cứng, các giao thức mạng, cơ sở hạ tầng, người sử dụng ... và các quan hệ ràng buộc lẫn nhau giữa các phần tử này.

❖ Hệ phần mềm (software system)?

- HPM được xem là phần tử chủ yếu trong một hệ thống thông tin vì chỉ bản thân nó mới có khả năng kết nối và điều khiển các phần tử khác cùng hoạt động với nhau. Như vậy, hệ thống thông tin là khái niệm bao trùm khái niệm hệ phần mềm.
- Đặc quan trọng của một hệ phần mềm là nó chỉ có thể hoạt động khi có sự tương thích với các thiết bị phần cứng, cơ sở hạ tầng, môi trường.

I MỘT SỐ ĐẶC TRƯNG CỦA HỆ THỐNG

- *Thành phần (component)* cũng còn gọi là hệ thống con được hiểu là một bộ phận đơn lẻ hay là hợp của một số bộ phận khác trong hệ thống.
- *Thành phần liên quan (interrelated components)* hay còn gọi là các thành phần kết nối mà hoạt động của nó có liên quan mật thiết với hoạt động của các thành phần khác.
- *Biên (boundary)* của hệ thống nhằm chỉ ra các thành phần bên trong mà chúng ta cần phải phát triển và thành phần bên ngoài không phải phát triển nhưng có liên quan đến hoạt động của hệ thống.
- *Môi trường (environment)* nhằm chỉ ra những gì bên ngoài biên và có ảnh hưởng đến hệ thống. Hệ thống tương tác với môi trường thông qua giao diện (interface) để nhận dữ liệu hay thông tin (dữ liệu đã được xử lý) và hiển thị kết quả.

I 1.2 CÁC KIỂU HỆ THỐNG THÔNG TIN

- **Hệ xử lý giao dịch:** Đây là một hệ thống có chức năng thu thập, xử lý, truyền đạt các thông tin, dữ liệu trong lĩnh vực kinh tế và thương mại. Ví dụ, hệ xử lý giao dịch mua bán hàng hay giao dịch ngân hàng, điều khiển thiết bị hay dây chuyền sản xuất, lập lịch, xử lý đơn đặt hàng...
- **Hệ cơ sở tri thức:** Dành cho các nhân viên tri thức và xây dựng dữ liệu nhằm giúp tổ chức, khám phá và tích hợp tri thức mới từ tri thức hiện thời vào nghiệp vụ của họ hay điều khiển luồng công việc. Ví dụ, các hệ thống hoạt động dựa trên tri thức, tự động hóa văn phòng, hệ xử lý ngôn ngữ, hệ thống tư vấn trong thương mại điện tử...
- **Hệ thông tin quản lý:** Dành cho các nhà quản lý trung gian phục vụ việc giám sát, điều khiển, ra quyết định và các hoạt động quản trị hay hỗ trợ ra các quyết định quan trọng với các yêu cầu về thông tin không rõ ràng. Ví dụ các hệ thông tin quản lý, hệ hỗ trợ quyết định, hệ quản lý bán hàng cần biết hàng tồn kho, ngân sách hàng năm để lập kế hoạch sản xuất, phân tích chi phí, phân tích giá cả/lợi nhuận, hệ thống tính cước và chăm sóc khách hàng ...
- **Hệ thống website:** là các hệ thống có nhiệm vụ cung cấp thông tin cho người dùng trên môi trường mạng Internet. Ví dụ các trang báo điện tử, trang dịch vụ bán vé máy bay ...

I 1.2 CÁC KIỂU HỆ THỐNG THÔNG TIN

- **Hệ thống thương mại điện tử**: là các hệ thống website đặc biệt phục vụ việc trao đổi mua bán hàng hoá, dịch vụ trên môi trường Internet. Hệ thống thương mại điện tử bao gồm cả các nền tảng hỗ trợ các giao thức mua bán, các hình thức thanh toán, chuyển giao hàng hoá...
- **Hệ thống điều khiển**: là các hệ thống phần mềm gắn với các thiết bị phần cứng hoặc các hệ thống khác nhằm mục đích điều khiển và giám sát hoạt động của thiết bị hay hệ thống đó. Hệ thời gian thực và hệ thống nhúng cũng có thể xem là các hệ thống thuộc lớp này.
- **Hệ thống chiến lược**: Dành cho các nhà quản lý giải quyết, vạch ra các chiến lược và các xu hướng lâu dài; so sánh khả năng của tổ chức với những thay đổi của môi trường bên ngoài và cơ hội xảy ra trong khoảng thời gian dài. Ví dụ, hệ hỗ trợ thực thi cho dự đoán ngân sách, xu hướng bán hàng trong các năm, kế hoạch hoạt động, kế hoạch về lợi nhuận, nhân lực.
- ...

| 1.3 CÁC CÁCH TIẾP CẬN PHÁT TRIỂN HỆ THỐNG THÔNG TIN

1.3.1 Các thành phần chủ yếu:

- *Phương pháp luận (methodology)*: là một loạt các bước có thể thực hiện lặp đi lặp lại mà chúng ta cần phải tuân theo để xây dựng sản phẩm phần mềm.
- *Mô hình hệ thống (modeling system)*: để xây dựng hay hiểu được sự hoạt động của bất kỳ hệ thống nào chúng ta cũng cần đến ngôn ngữ sử dụng để mô hình hóa hệ thống đó.
- *Kỹ thuật (technique)*: Các kỹ thuật liên quan đến các công việc như phỏng vấn, điều tra ... để thu thập yêu cầu khách hàng; kỹ thuật lập kế hoạch và quản lý dự án; kỹ thuật xây dựng các biểu đồ để mô hình chức năng hệ thống.
- *Công nghệ (technology)*: Các công nghệ nền tảng hỗ trợ phát triển phần mềm như J2EE, .NET ... hay các Framework như Spring, EF ..
- *Công cụ (tool)*: Các công cụ là các phần mềm hỗ trợ cho phát triển như Netbean, Eclipse, Visual Studio...

| 1.3.2 Một số khái niệm cho nguyên lý phát triển phần mềm

- *Phân rã (decomposition)* là quá trình phân chia hệ thống thành các thành phần nhỏ hơn mà đôi khi còn gọi là hệ thống con (subsystem) và các hệ thống con này cũng có thể phân chia thành các hệ thống con nhỏ hơn nữa.
- *Môđun hóa (modularity)* là hệ quả trực tiếp của việc phân rã. Nó liên quan đến việc phân chia hệ thống thành các môđun có kích thước tương đối đồng đều nhau. Các môđun có thể giúp chúng ta hiểu hệ thống dễ dàng và hơn nữa giúp dễ dàng thiết kế và xây dựng lại.
- *Kết nối (coupling)* nhằm chỉ sự phụ thuộc giữa các hệ thống con. Nguyên lý thiết kế cho rằng các hệ thống con nên phụ thuộc vào nhau một cách *lỏng lẻo* để sự đình trệ của một thành phần không ảnh hưởng đến hoạt động của các thành phần khác.
- *Kết dính (cohesion)* liên quan đến mức độ kết nối của các thành phần bên trong một hệ thống con. Nguyên lý thiết kế cho rằng các yếu tố bên trong một thành phần nên kết dính chặt với nhau.

| 1.3.3 Phân tích và thiết kế hệ thống thông tin ?

- *Phân tích và thiết kế hệ thống thông tin* (information system analysis and design) là một tiến trình phức tạp, có tổ chức mà các thành viên của nhóm xây dựng và tổ chức khách hàng cần phải tuân theo để phát triển và bảo trì hệ thống.
- Việc phân tích và thiết kế các hệ thống thông tin phải được định hình từ chính yêu cầu của tổ chức khách hàng.
- Hệ thống thông tin liên quan đến nhiều khía cạnh như các hệ phần cứng, các hệ phần mềm, cơ sở hạ tầng mạng, tài liệu huấn luyện sử dụng...sản phẩm quan trọng nhất của kết quả quá trình phân tích và thiết kế hệ thống là *phần mềm ứng dụng*.
- Thuật ngữ *phát triển phần mềm* (software development) nhằm chỉ một quá trình hay các pha mà chúng ta phải trải qua trong việc tạo ra phần mềm thực thi được.

| 1.3.4 Các phương pháp phát triển phần mềm

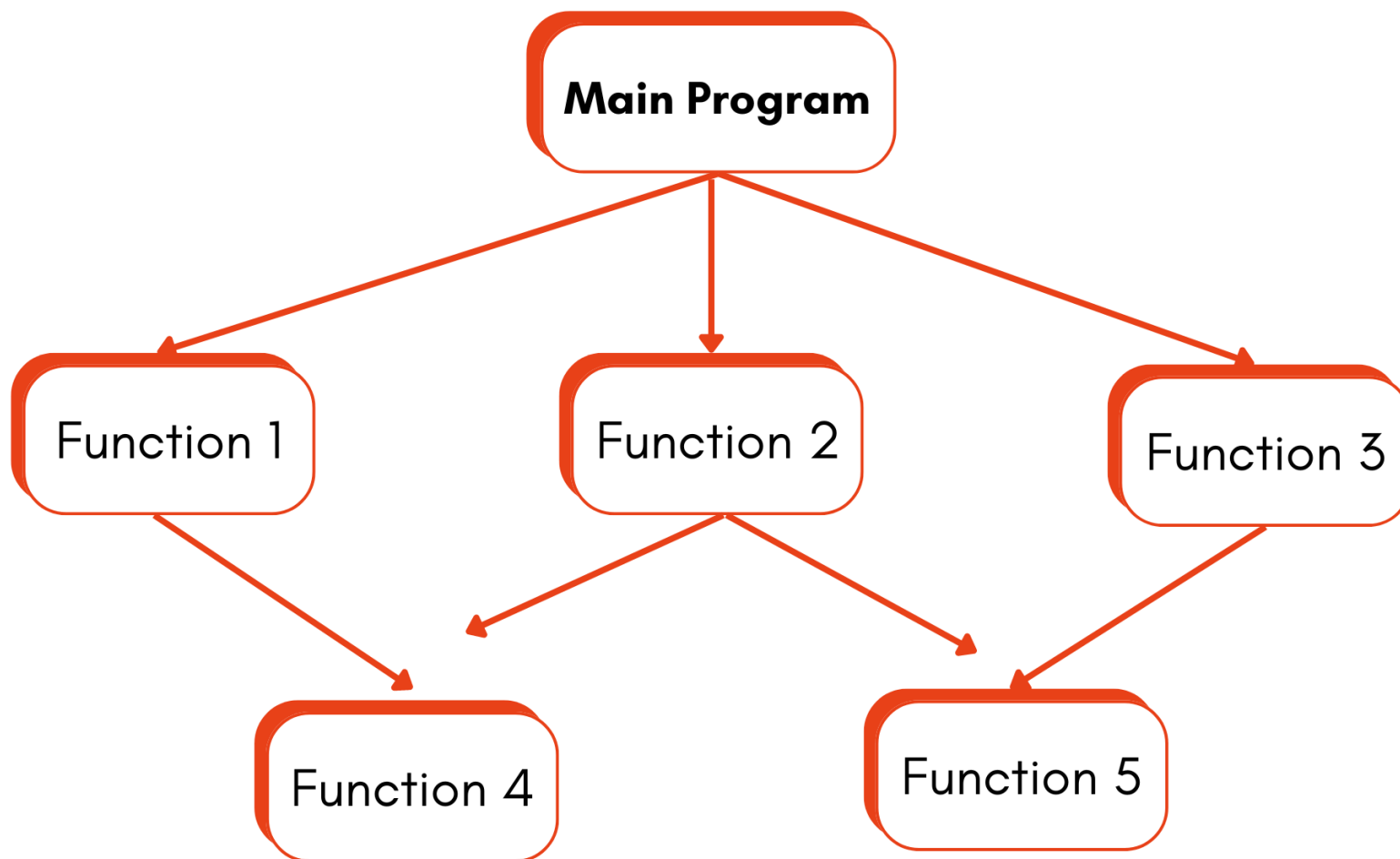
Một số phương pháp phổ biến:

- Phương pháp phát triển hướng cấu trúc (POP).
- Phương pháp phát triển hướng đối tượng (OOP).
- Phương pháp phát triển hướng thành phần (COP).
- Phương pháp phát triển hướng dịch vụ (SOA).

| Phương pháp phát triển hướng cấu trúc

- Ra đời vào những năm 70, 80 của thế kỷ XX.
- Phân chia chương trình chính thành nhiều chương trình con, mỗi chương trình con nhằm đến thực hiện một công việc xác định.
- Thiết kế dựa trên một trong hai hướng: dữ liệu hoặc hành động.
- Cách thức thực hiện là phương pháp thiết kế từ trên xuống (**top-down**). Phương pháp này tiến hành phân rã hệ thống thành các hệ thống nhỏ hơn, rồi tiếp tục phân rã các hệ thống con cho đến khi có được các thành phần có thể cài đặt được bằng các hàm của C, Pascal.
- Ưu điểm: là tư duy phân tích thiết kế rõ ràng, chương trình sáng sủa dễ hiểu.
- Nhược điểm: không phù hợp trong phát triển các hệ phần mềm lớn và đặc biệt là kém hiệu quả trong việc sử dụng lại như yêu cầu trong công nghiệp phần mềm ngày nay.

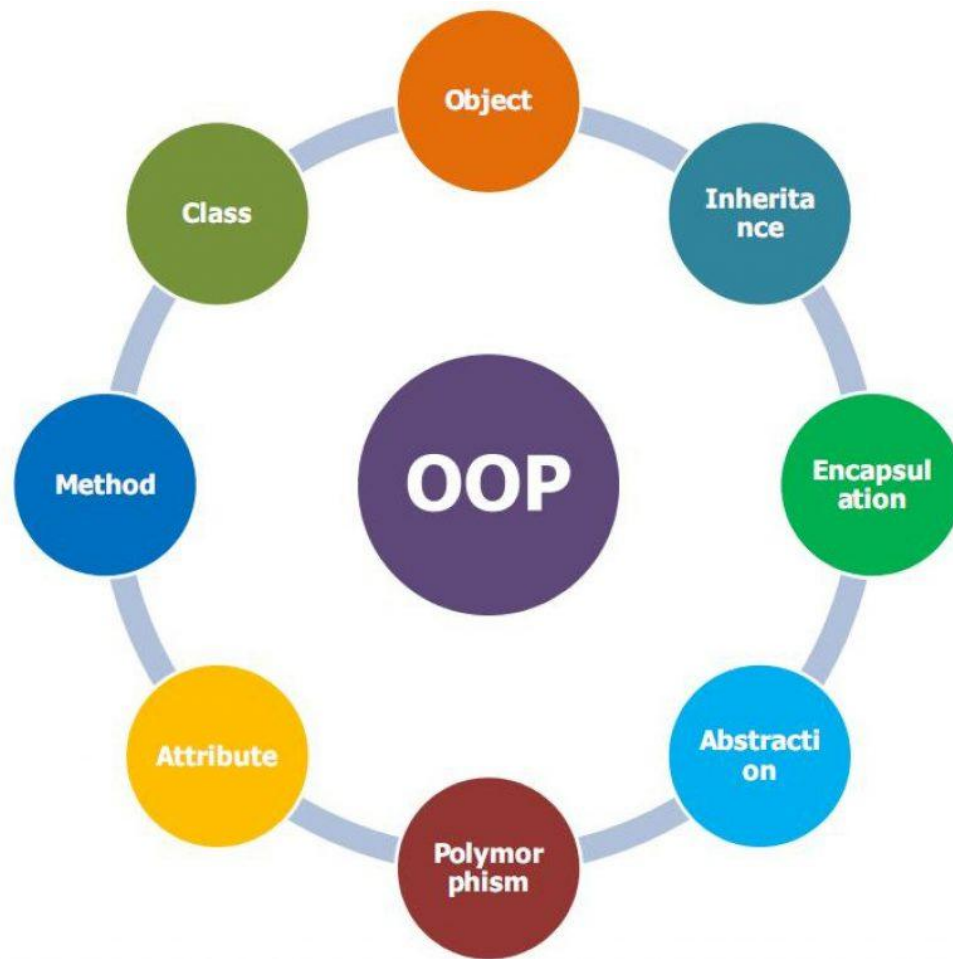
| Phương pháp phát triển hướng cấu trúc



| Phương pháp phát triển hướng đối tượng (OOP)

- Ra đời vào những năm 90 của thế kỷ XX.
- Tập trung vào cả hai khía cạnh của hệ thống là dữ liệu và hành động.
- Hệ thống được chia tương ứng thành các thành phần nhỏ gọi là các **đối tượng**, mỗi đối tượng bao gồm đầy đủ cả dữ liệu và hành động liên quan đến đối tượng đó.
- Các đối tượng trong một hệ thống tương đối độc lập với nhau và phần mềm được xây dựng bằng cách kết hợp các đối tượng đó lại với nhau thông qua các mối quan hệ và tương tác giữa chúng.
- Được cài đặt bằng các ngôn ngữ hướng đối tượng: C++, C#, Java...
- Cách tiếp cận **hướng đối tượng** đã trở thành chuẩn mực của công nghiệp phần mềm và đã được sử dụng rộng rãi cho phát triển các hệ thống phần mềm trong quản lý các doanh nghiệp, thương mại điện tử, các hệ thời gian thực, các hệ thống thông minh...

| Phương pháp phát triển hướng đối tượng (OOP)

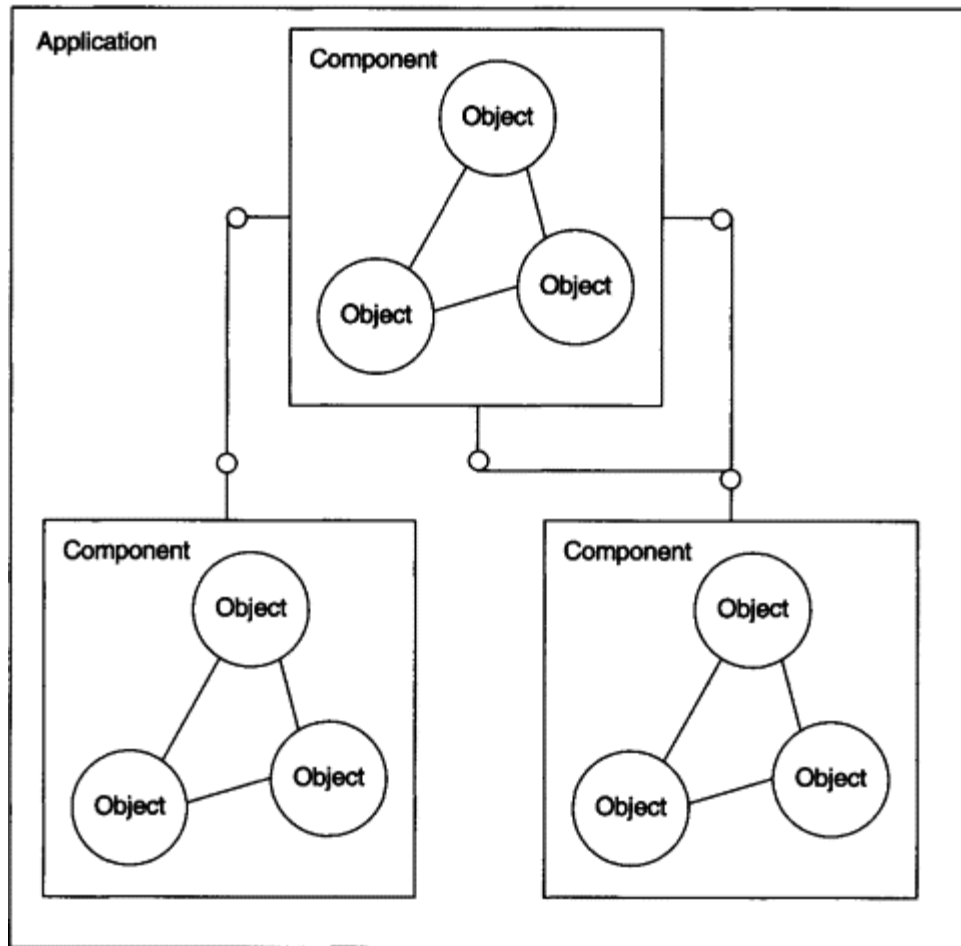


| Phương pháp phát triển hướng thành phần (COP)

- Phân rã kiến trúc thành các **thành phần** chức năng riêng biệt hoặc các thành phần tiếp xúc với giao diện chuẩn qua kết nối.
- Một **thành phần** là một phần của một hệ thống gần như độc lập và có khả năng thay thế. Nó thoả mãn một số chức năng cụ thể trong ngữ cảnh của kiến trúc đã xác định.
- Các công nghệ thành phần như: JavaBeans, EJB, COM, DCOM, .Net, ...
- Ưu điểm: giảm thiểu phức tạp, quản lý thay đổi và tái sử dụng.

| Phương pháp phát triển hướng thành phần (COP)

Component-Based Development



Applications in the Component Architecture

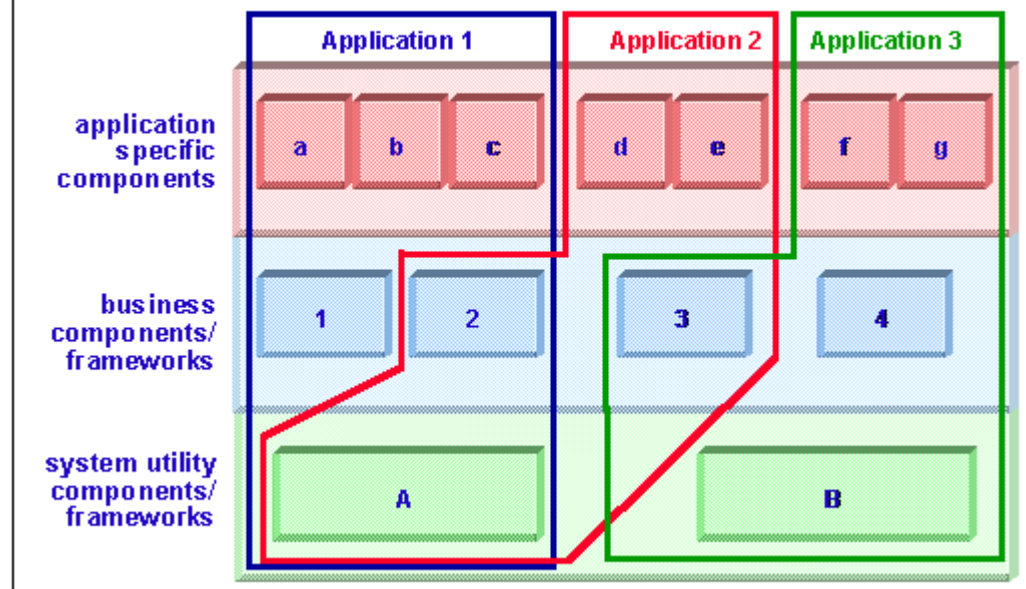


Figure 2.6



| Phương pháp phát triển hướng dịch vụ (SOA)

- Service-Oriented Architecture (SOA) được định nghĩa là kiểu kiến trúc trong phần mềm bao gồm nhiều thành phần đơn lẻ. Chúng tập trung lại và tạo thành những dịch vụ, mỗi dịch vụ thực hiện một nhiệm vụ, quy trình khác nhau.
- Thành phần được kết nối qua cổng giao tiếp, chúng có tính kế, tương tác với nhau mà không cần quan tâm đến việc mình được phát triển trên nền tảng công nghệ nào. Điều này có tác dụng giúp hệ thống mở rộng hơn và tích hợp một cách dễ dàng.
- Ưu điểm: tái sử dụng, tính linh hoạt cao, dễ dàng mở rộng, bảo trì.



| Mô hình hóa hướng đối tượng

❖ Mô hình?

- Là một dạng trừu tượng hóa/một hình ảnh/một biểu diễn của một hệ thống thực, được diễn tả: ở một mức độ trừu tượng hóa nào đó, theo một quan điểm/góc nhìn nào đó, bởi một hình thức diễn tả hiểu được nào đó (như văn bản, đồ thị, phương trình,...).

❖ Mô hình hóa

- Dùng mô hình để nhận thức và diễn tả một hệ thống.

❖ Mục đích của Mô hình hóa:

- Để hiểu hệ thống.

-> Quá trình phân tích và thiết kế Hệ thống cũng được gọi là quá trình mô hình hóa Hệ thống.

| Phương pháp Mô hình hóa

Kết hợp 3 thành phần:

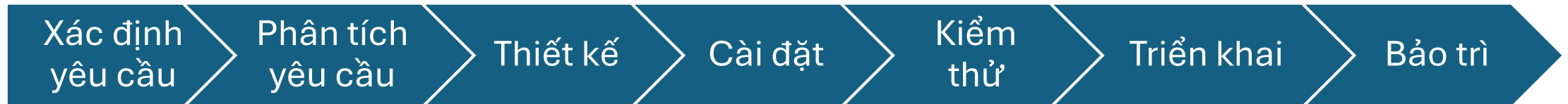
- **Hệ ký hiệu** (notation): Các khái niệm và mô hình.
- **Một tiến trình** (process): Các bước cần tiến hành, các sản phẩm (tài liệu, mô hình) qua từng giai đoạn, cách điều hành tiến trình, cách đánh giá chất lượng.
- **Công cụ hỗ trợ**: Phần mềm hỗ trợ cho quá trình **Mô hình hóa**, có khả năng:
 - Sản sinh các mô hình và biểu đồ.
 - Biến đổi và điều chỉnh nhanh các mô hình và biểu đồ.
 - Kiểm tra cú pháp, chặt chẽ, đầy đủ.
 - Kiểm thử và đánh giá.
 - Mô phỏng thực hiện mô hình.

| Xu hướng của Mô hình hóa

- Mô hình hóa hướng chức năng (từ 1970):
 - Lấy chức năng làm đơn vị phân rã Hệ thống.
 - Phù hợp với các phương pháp lập trình hướng thủ tục.
- Mô hình hóa hướng đối tượng (từ 1990):
 - Lấy đối tượng làm đơn vị phân rã Hệ thống.
 - Phù hợp với các phương pháp lập trình hướng đối tượng.

| 1.4 CÁC PHA CƠ BẢN CỦA PHÂN TÍCH VÀ THIẾT KẾ

Có nhiều mô hình phát triển phần mềm khác nhau như mô hình thác nước, xoắn ốc, agile, scrum,... nhưng cơ bản có các pha sau đây:



| Xác định yêu cầu

Đây là pha đầu tiên của quá trình phát triển phần mềm. Nó bao gồm hai giai đoạn:

- **Mô hình hóa nghiệp vụ:** liên quan đến việc hiểu nghiệp vụ hiện thời với ngữ cảnh cụ thể của nó mà dự án cần phát triển. Nó có thể là mô hình quản lý đang hoạt động hiện thời hay phần mềm đang sử dụng hiện thời...
- **Mô hình hóa yêu cầu hệ thống:** xác định được yêu cầu thực sự của khách hàng cho hệ thống định xây dựng, hiểu được khách hàng cần gì để biết ta sẽ phải làm gì và không cần làm gì, càng chi tiết và càng rõ ràng càng tốt.

| Phân tích yêu cầu

- Phân tích có nghĩa là ta phải xét xem ta đang phải đối mặt với vấn đề gì.
- Đi sâu vào bản chất, hiểu rõ những thực thể liên quan, những tính chất và những mối quan hệ bên trong của chúng. Từ đó mới biết được ta đã thực sự hiểu về sản phẩm mà khách hàng yêu cầu hay chưa vì điều này liên quan đến khách hàng và người dùng cuối.

| Thiết kế

Thiết kế là đưa ra cách thức giải quyết vấn đề, có 2 giai đoạn:

- Thiết kế hệ thống (**system design**) hay thiết kế kiến trúc (**architecture design**): phân rã một hệ thống thành các hệ thống con liên quan nhau về mặt logic và hệ thống con liên quan nhau về vật lý dựa trên các công nghệ, thiết bị máy móc đã lựa chọn.
- Thiết kế các hệ thống con (**subsystem design**) hay thiết kế chi tiết (**detailed design**): cách quyết định chuyển hệ thống con liên quan nhau về mặt logic thành mã thế nào.

| CÁC PHA KHÁC

- **Cài đặt:** viết mã cho các hệ thống con, viết phần thân các phương thức trong lớp thiết kế. Tích hợp thành hệ thống lớn theo yêu cầu.
- **Kiểm thử:** Kiểm thử xem sản phẩm đã đáp ứng các yêu cầu của khách hàng chưa. Cách tốt nhất thường được tiến hành làm là thực hiện các kiểm thử trong suốt quá trình phát triển phần mềm để đảm bảo chất lượng.
- **Triển khai:** Cài đặt phần mềm đưa vào sử dụng, huấn luyện và đào tạo cho người dùng.
- **Bảo trì:** Sau khi hệ thống được triển khai, có thể sẽ có nhiều lỗi hệ thống hay cũng có thể khách hàng yêu cầu sửa đổi, nâng cấp...vì vậy cần có quá trình bảo trì hệ thống.

| 1.5. CÁC CÔNG NGHỆ CHO PHÁT TRIỂN CÁC KIỂU HỆ PHẦN MỀM

Chọn công nghệ phát triển phần mềm phụ thuộc vào nhiều yếu tố

- Môi trường chạy phần mềm (Web, App, Windows Form, Thiết bị ...)
- Môi trường triển khai phần mềm (Linux, Windows, On Premises, Cloud ...)
- Hạ tầng phần cứng.
- Tải hệ thống, số người truy cập đồng thời.
- Đặc thù của phần mềm: ví dụ như hệ xử lý giao dịch cần quản lý tốt các transaction.
- Trình độ của nhân lực.
- Yêu cầu của Khách hàng.
- Các yêu cầu khác như về bảo mật dữ liệu, tích hợp hệ thống, ...

I MỘT SỐ CÔNG NGHỆ PHÁT TRIỂN PHẦN MỀM PHỔ BIẾN

Ngôn ngữ lập trình	Front End	Back End	App	Middleware	Database
• .Net, .Net Core (C#, VB.NET) • Java • Python • Perl • Erlang • Golang • PHP • ...	• HTML5 • CSS3 • JavaScript • TypeScript • ReactJS • NextJS • VueJS • ...	• Web API • Spring Boot • FastAPI • Slim Framework • NodeJS • ...	• Android: Java, Kotlin • iOS: Swift, Objective-C • Cross Platform: Flutter, React Native, Ionic • ...	• Laravel • Django • Ruby on Rails • ExpressJS • ...	• SQL: Oracle, SQL Server, MySQL, MariaDB ... • NoSQL: MongoDB, Cansandra, Riak • ...

I 1.6 CÁC QUY TRÌNH PHÁT TRIỂN PHẦN MỀM

❖ Quy trình PTPM?

- Quy trình phát triển phần mềm (Software Development Life Cycle - SDLC) là một tập hợp các bước được thực hiện để tạo ra phần mềm. SDLC giúp đảm bảo rằng phần mềm được phát triển một cách hiệu quả và đáp ứng được nhu cầu của người dùng.

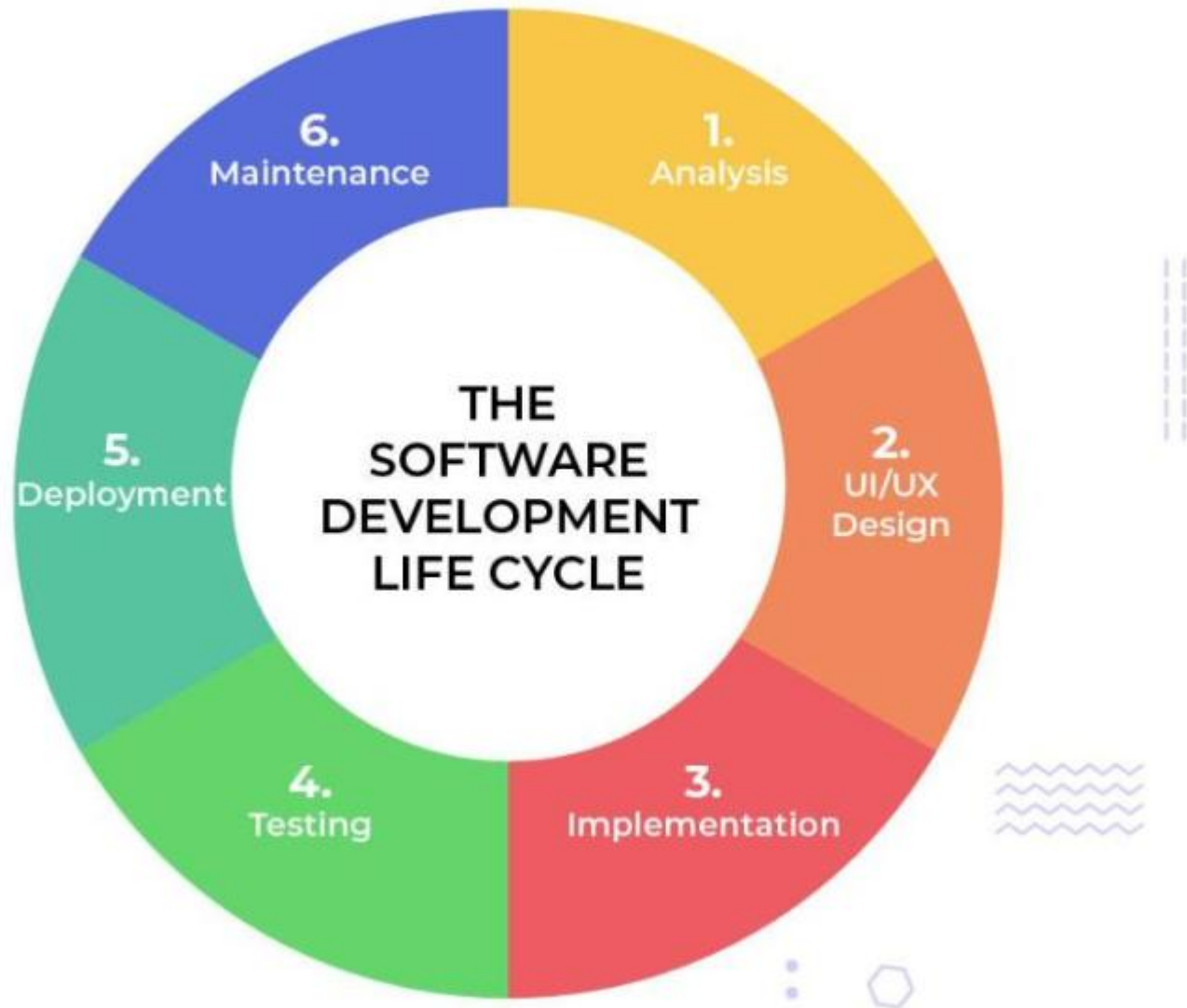
❖ Có nhiều quy trình PTPM

- Vd: Mô hình thác nước (Waterfall), Nguyên mẫu (Prototyping), Xoắn ốc (Spiral), Linh hoạt (Agile) ...
- Không tồn tại một quy trình PTPM lý tưởng duy nhất phù hợp cho mọi bài toán mà nó phải phù hợp với yêu cầu thực tế.

❖ Các yếu tố lựa chọn quy trình PTTM

- Kiểu của hệ thống phần mềm cần được xây dựng: phát triển mới, nâng cấp...
- Các yêu cầu là xác định hay chưa xác định, còn thay đổi.
- Quy mô dự án, nguồn kinh phí, nhân lực.

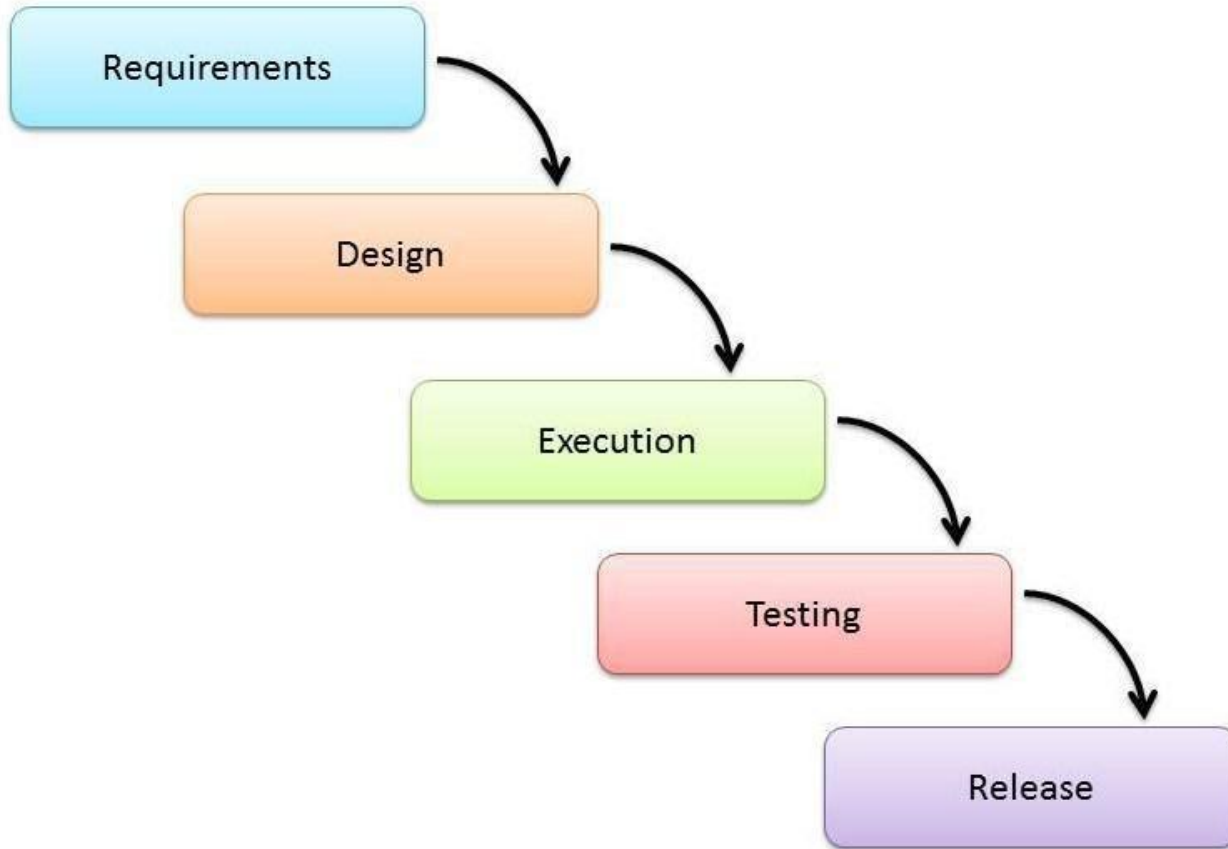
| Các hoạt động cơ bản của Quy trình PTPM



| Một số quy trình PTPM thông dụng

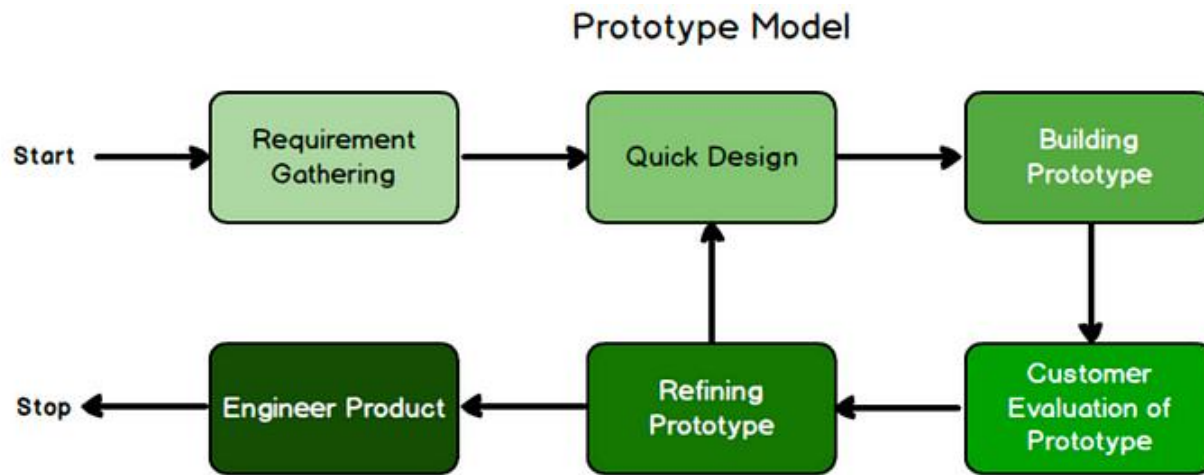
- Mô hình thác nước (Waterfall model)
- Mô hình nguyên mẫu (Prototyping model)
- Mô hình lặp (Iterative model)
- Mô hình xoắn ốc (Spiral model)
- Mô hình hợp nhất (Rational Unified Process)
- Mô hình chữ V (V-Model)
- Mô hình linh hoạt (Agile model)
- Mô hình Scrum (Scrum Model)

| Mô hình thác nước (Waterfall model)



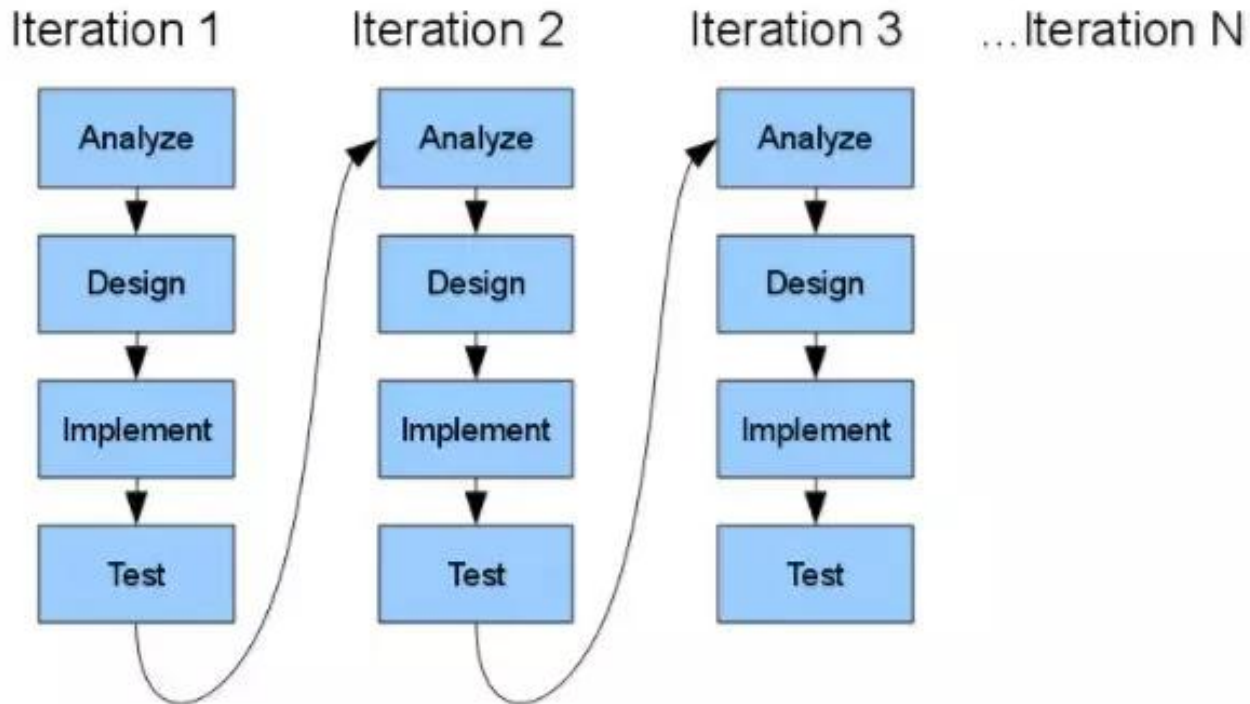
- Waterfall là một trong những phương pháp lâu đời và có cấu trúc cổ điển nhất.
 - Sắp xếp các giai đoạn theo trình tự, các giai đoạn sau sẽ phụ thuộc vào toàn bộ thông tin và kết quả của giai đoạn trước. Giống như thác nước chảy, quy trình sẽ không thể quay ngược về giai đoạn trước đó.
 - **Ưu điểm:** Dễ dàng quản lý dự án và đưa ra kết quả rõ ràng ở cuối mỗi giai đoạn.
 - **Nhược điểm:** Ít khả năng thay đổi sau khi một giai đoạn được coi là hoàn thành, những thay đổi này có thể gây ảnh hưởng tới thời gian, chi phí và chất lượng của phần mềm
- > Phù hợp với các dự án triển khai phần mềm nhỏ.

| Mô hình nguyên mẫu (Prototyping model)



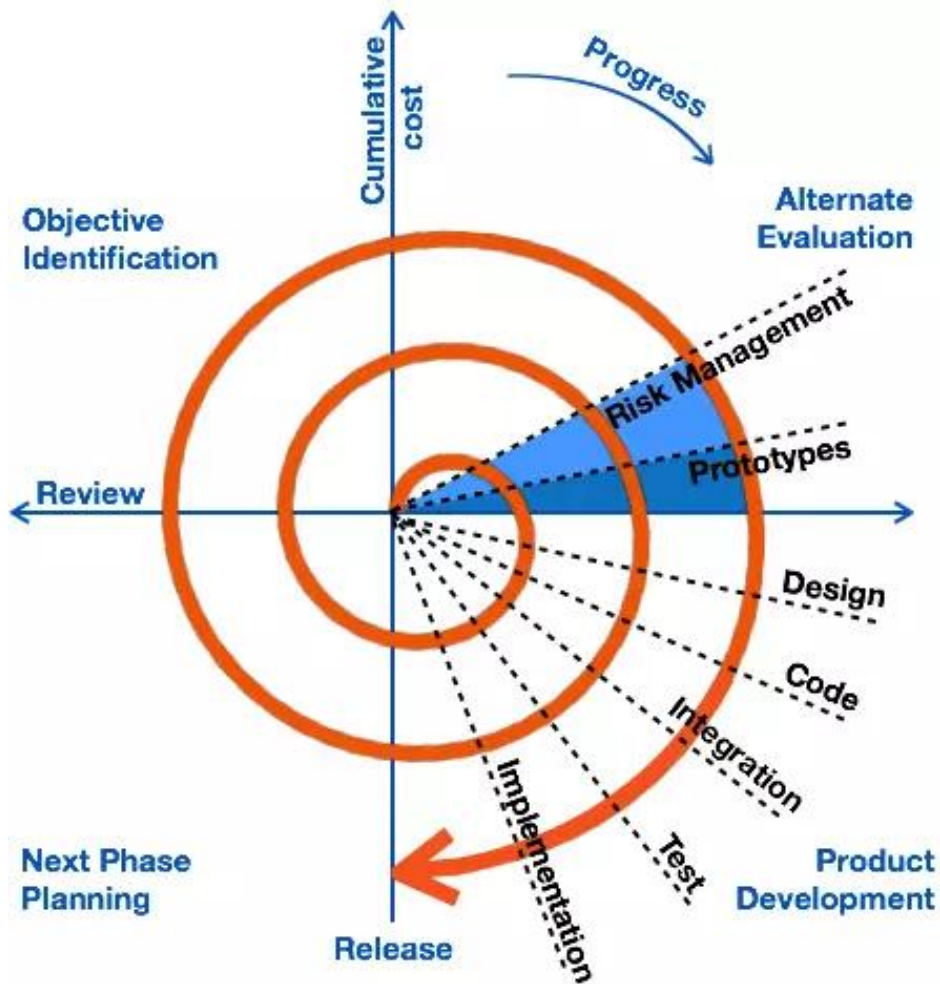
- Prototype là một mô hình phát triển phần mềm được phát triển dựa trên các yêu cầu hệ thống. Dựa vào bản prototype mà khách hàng có cái nhìn tổng quan về hệ thống thực tế.
- **Ưu điểm:**
 - Người dùng tích cực tham gia vào quá trình phát triển phần mềm từ đó sẽ hiểu rõ hơn về hệ thống đang được phát triển.
 - Các lỗi, vấn đề có thể được phát hiện từ sớm.
- **Nhược điểm:**
 - Không có kế hoạch ngay từ đầu.
 - Nếu bản prototype mà khách hàng không hài lòng phải làm lại gây tốn thời gian và công sức, chi phí và chất lượng của phần mềm.
- **Nên được dùng khi:**
 - Không xác định được chính xác yêu cầu.
 - Cần có nhiều tương tác với người dùng cuối.
 - Cần kiểm tra, đánh giá công nghệ.

| Mô hình lặp (Iterative model)



- Thay vì bắt đầu với những yêu cầu đầy đủ, mô hình lặp lại sẽ triển khai một tập hợp các yêu cầu phần mềm, sau đó kiểm thử và đánh giá, xác định các yêu cầu kế tiếp.
- Mỗi giai đoạn hay lần lặp lại sẽ là một phiên bản mới của phần mềm. Quá trình sẽ được lặp lại đến khi có được một hệ thống hoàn chỉnh.
- **Ưu điểm:** Xác định dễ dàng các rủi ro có thể xảy ra khi các yêu cầu có thể thay đổi giữa các lần lặp.
- **Nhược điểm:** Chu kỳ lặp lại có thể dẫn đến thay đổi phạm vi và tài nguyên nhanh chóng bị tiêu tốn.

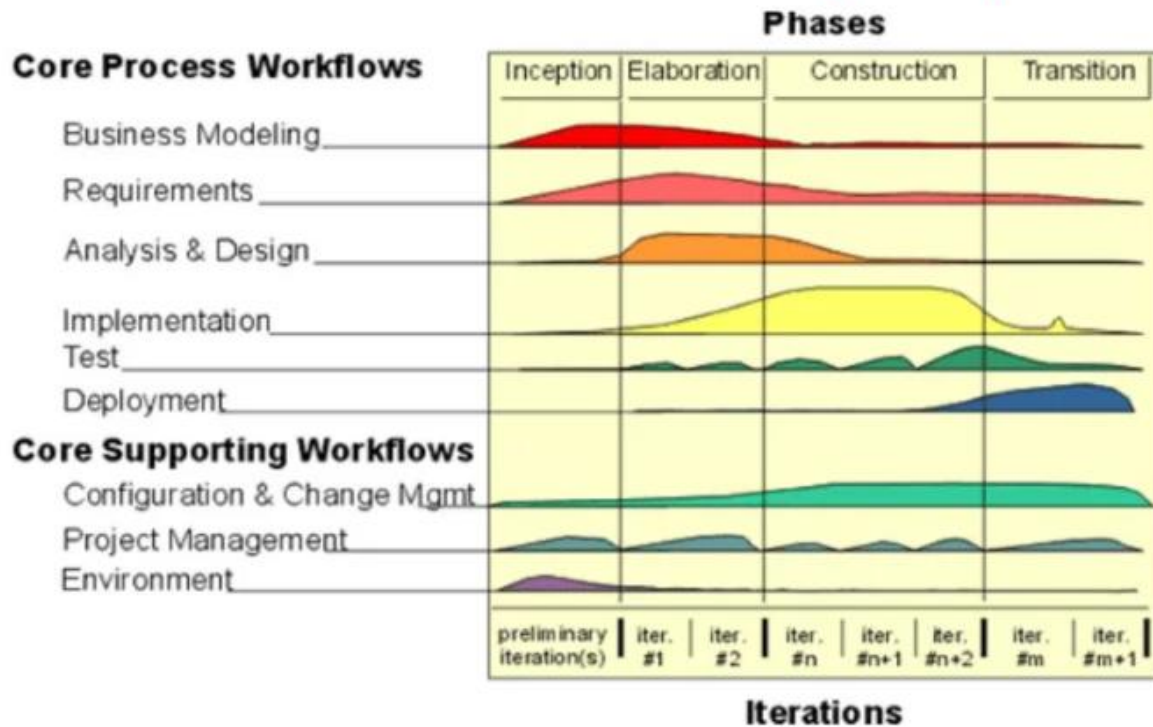
| Mô hình xoắn ốc (Spiral model)



- Mô hình xoắn ốc kết hợp các ưu điểm của mô hình waterfall và mô hình iterative, trong đó các giai đoạn được thử nghiệm lặp đi lặp lại và liên tục cải tiến cho đến khi hoàn thành.
 - Mô hình này có thể sử dụng để đảm bảo phát hành dần dần và từng bước cải thiện phần mềm bằng cách xây dựng nguyên mẫu ở mỗi giai đoạn.
 - Chú trọng vào phân tích nguy cơ rủi ro.
 - **Ưu điểm:** Phù hợp với những dự án lớn, phức tạp, có yêu cầu phải thay đổi thường xuyên, giảm thiểu rủi ro.
 - **Nhược điểm:** Mô hình có chi phí lớn, tốn kém, không phù hợp với các dự án nhỏ và phạm vi hạn chế.
- > Nên dùng khi việc đánh giá các chi phí và rủi ro là quan trọng.

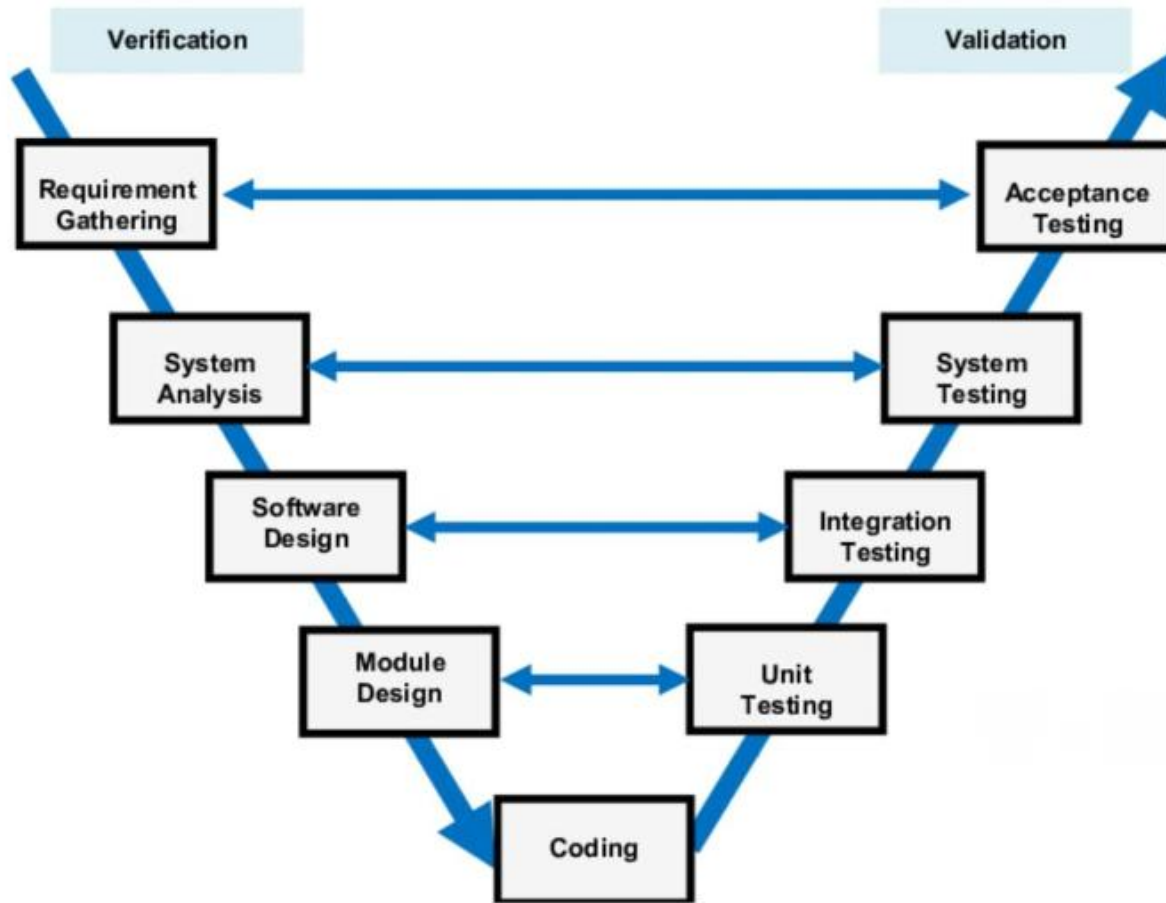
| Mô hình hợp nhất (Rational Unified Process)

Rational Unified Process (RUP)



- Quy trình hợp nhất là sự mở rộng của tiến trình xoắn ốc, nhưng hình thức hơn và chặt chẽ hơn
- Quy trình bao gồm bốn giai đoạn chính và đan xen nhiều dòng hoạt động như là: Mô hình hoá nghiệp vụ, phân tích yêu cầu, phân tích và thiết kế, cài đặt, thử nghiệm triển khai, ... Mỗi giai đoạn được hình thành từ những bước lặp.
- **Khởi tạo (inception):**
 - Thiết lập phạm vi dự án, các điều kiện ràng buộc.
 - Xác định chi phí, thời gian của dự án, đánh giá rủi ro.
- **Chi tiết hóa (elaboration):**
 - Phân tích chi tiết hơn về hệ thống.
 - Xây dựng nền kiến trúc nền tảng hệ thống.
- **Xây dựng (construction):**
 - Tập trung vào thiết kế và cài đặt hệ thống.
- **Chuyển giao (transition):**
 - Cài đặt, chuyển giao hệ thống cho người dùng cuối.

| Mô hình chữ V (V-Model)



- Là mô hình tuyến tính, mỗi giai đoạn đều có một lần chạy thử nghiệm. Có nghĩa là tiến hành song song 2 hoạt động: phát triển và kiểm thử.
- Mô hình có tính kỷ luật cao. Giai đoạn tiếp theo chỉ có thể bắt đầu khi giai đoạn trước hoàn thành.
- Việc tester được tham gia ngay từ đầu nên lỗi được tìm ra từ sớm để khắc phục.
- **Nên áp dụng khi:**
 - Các dự án yêu cầu không có lỗi và thời gian chết như các phần mềm dùng trong y tế hay phần mềm quản lý chuyến bay.
 - Các dự án ngắn. Công nghệ không có sự thay đổi và được nhóm phát triển dự án hiểu rõ.

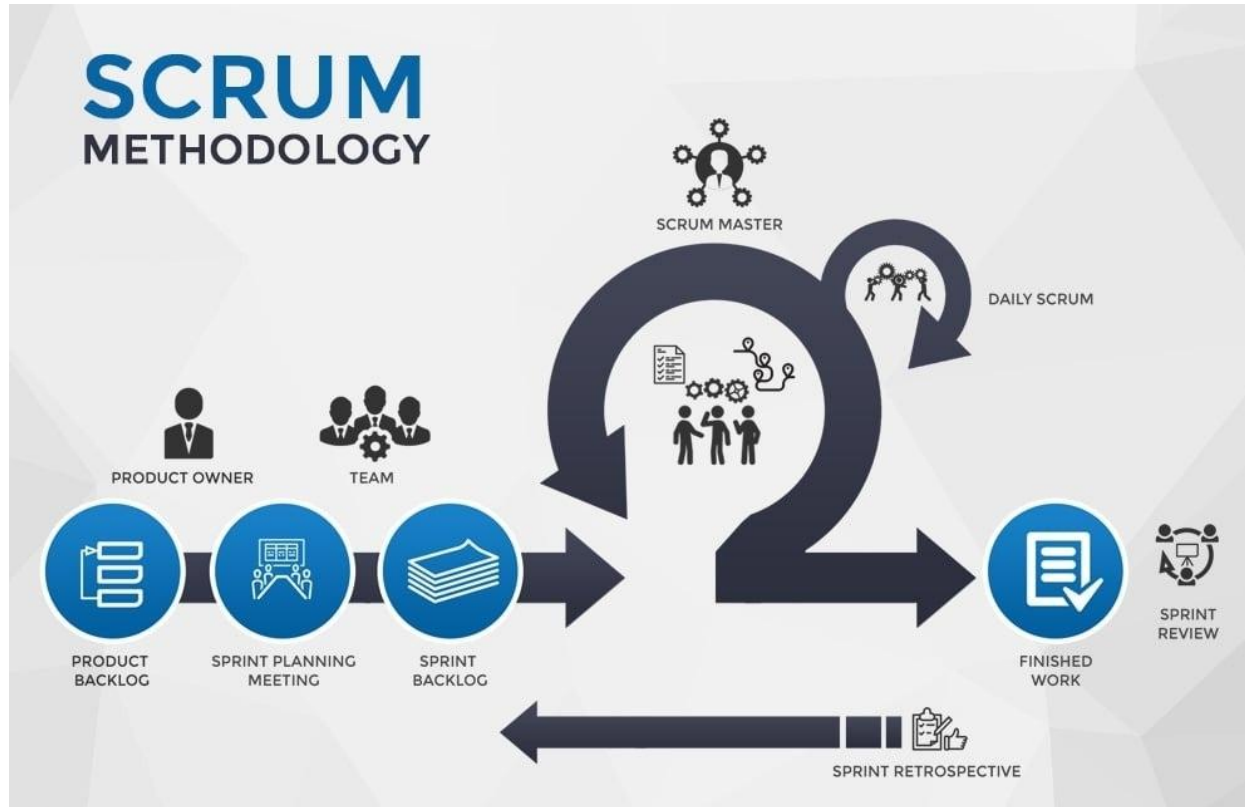
| Mô hình linh hoạt (Agile model)

AGILE



- Mô hình linh hoạt sắp xếp các giai đoạn SDLC thành nhiều chu trình phát triển. Mô hình tạo ra các bản phát hành liên tục, mỗi bản sẽ có những thay đổi nhỏ, phát triển hơn so với bản phát hành trước. Mỗi lần phát hành, sản phẩm đều được kiểm thử đánh giá một cách chi tiết.
- Đây là mô hình được đánh giá mang lại hiệu quả nhiều nhất.
- **Ưu điểm:** Các chu trình phát triển nhanh chóng giúp các nhóm xác định và giải quyết vấn đề trong các dự án phức tạp ngay từ đầu. Thu thập các ý kiến phản hồi từ người dùng trong suốt dự án để cải thiện chất lượng sản phẩm.
- **Nhược điểm:** Phụ thuộc quá nhiều vào ý kiến phản hồi từ người dùng có thể dẫn đến sự thay đổi quá mức phạm vi dự án.

| Mô hình Scrum (Scrum Model)



- Mô hình Scrum là một mô hình quy trình phát triển phần mềm thuộc dạng Agile.
- Gồm ba vai trò chính: Product Owner (chủ sản phẩm), Scrum Master (quản lý dự án) và Development Team (nhóm phát triển). Mỗi dự án được chia thành các sprint (chu kỳ), thường kéo dài từ hai tuần đến một tháng.
- Trong mỗi sprint, nhóm sẽ hoàn thành một số tính năng của sản phẩm theo danh sách ưu tiên (backlog) do Product Owner xác định.
- Mỗi ngày, nhóm phát triển sẽ tổ chức một cuộc họp ngắn gọn để báo cáo tiến độ. Cuối mỗi sprint, nhóm phát triển sẽ thực hiện kiểm thử, trình bày sản phẩm cho khách hàng và người dùng để nhận phản hồi và cải tiến.
- Mô hình Scrum giúp tăng hiệu quả làm việc của nhóm, tạo ra sản phẩm chất lượng cao và thỏa mãn khách hàng.

